



ÉCOLE NATIONALE SUPÉRIEURE
D'INFORMATIQUE ET D'ANALYSE DES SYSTÈMES
- RABAT

ARTIFICIAL INTELLIGENCE

Linear regression : Predicting house
price case study

Élève :
RAISS IMANE

Encadrant :
Pr.

Année Universitaire 2025-2026

Chapitre 1

Case Study - Boston Housing Dataset

Question #1:

Is it your turn to deal with data preparation, such as selecting appropriate features to employ in order to obtain the best model?

```
from sklearn.datasets import fetch_california_housing
```

```
housing = fetch_california_housing()  
X, y = housing.data, housing.target
```

```
from sklearn.datasets import fetch_california_housing  
housing = fetch_california_housing()
```

```
print(housing.keys())
```

```
dict_keys(['data', 'target', 'frame', 'target_names', 'feature_names', 'DESCR'])
```

```
#Find features and target  
x = housing.data  
y = housing.target
```

```
▶ #find features name  
columns = housing.feature_names  
columns
```

```
... ['MedInc',  
     'HouseAge',  
     'AveRooms',  
     'AveBedrms',  
     'Population',  
     'AveOccup',  
     'Latitude',  
     'Longitude']
```

```
▶ print(housing.DESCR)
```

```
... .. _california_housing_dataset:
```

```
California Housing dataset  
-----
```

```
**Data Set Characteristics:**
```

```
:Number of Instances: 20640
```

```
:Number of Attributes: 8 numeric, predictive attributes and the target
```

```
:Attribute Information:
```

- MedInc median income in block group
- HouseAge median house age in block group
- AveRooms average number of rooms per household
- AveBedrms average number of bedrooms per household
- Population block group population
- AveOccup average number of household members
- Latitude block group latitude
- Longitude block group longitude

```
:Missing Attribute Values: None
```

```
This dataset was obtained from the StatLib repository.
```

```
https://www.dcc.fc.up.pt/~ltorgo/Regression/cal\_housing.html
```

```
The target variable is the median house value for California districts,  
expressed in hundreds of thousands of dollars ($100,000).
```

```
This dataset was derived from the 1990 U.S. census, using one row per census  
block group. A block group is the smallest geographical unit for which the U.S.  
Census Bureau publishes sample data (a block group typically has a population  
of 600 to 3,000 people).
```

```
# import pandas
import pandas as pd
```

```
▶ # Create dataframe of the boston data
housing_df = pd.DataFrame(housing.data)
housing_df.head(10)
```

```
...      0      1      2      3      4      5      6      7
0  8.3252  41.0  6.984127  1.023810  322.0  2.555556  37.88 -122.23
1  8.3014  21.0  6.238137  0.971880  2401.0  2.109842  37.86 -122.22
2  7.2574  52.0  8.288136  1.073446  496.0  2.802260  37.85 -122.24
3  5.6431  52.0  5.817352  1.073059  558.0  2.547945  37.85 -122.25
4  3.8462  52.0  6.281853  1.081081  565.0  2.181467  37.85 -122.25
5  4.0368  52.0  4.761658  1.103627  413.0  2.139896  37.85 -122.25
6  3.6591  52.0  4.931907  0.951362  1094.0  2.128405  37.84 -122.25
7  3.1200  52.0  4.797527  1.061824  1157.0  1.788253  37.84 -122.25
8  2.0804  42.0  4.294118  1.117647  1206.0  2.026891  37.84 -122.26
9  3.6912  52.0  4.970588  0.990196  1551.0  2.172269  37.84 -122.25
```

```
housing_df.shape
```

```
(20640, 8)
```

```
# Create dataframe
y_df = pd.DataFrame(y)
y_df.head()
```

```
...
0
0 4.526
1 3.585
2 3.521
3 3.413
4 3.422
```

```
housing_df.columns=columns
housing_df.head()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25

```
# check the shape of the DataFrame (rows, columns)
housing_df.shape
```

```
(20640, 8)
```

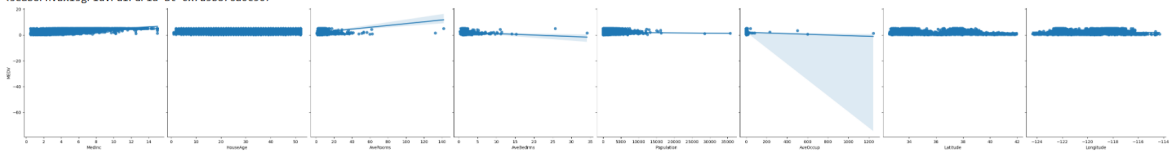
```
housing_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   MedInc          20640 non-null  float64
1   HouseAge        20640 non-null  float64
2   AveRooms        20640 non-null  float64
3   AveBedrms       20640 non-null  float64
4   Population      20640 non-null  float64
5   AveOccup        20640 non-null  float64
6   Latitude        20640 non-null  float64
7   Longitude       20640 non-null  float64
dtypes: float64(8)
memory usage: 1.3 MB
```

```
# conventional way to import seaborn
import seaborn as sns
# allow plots to appear within the notebook
%matplotlib inline
housing_df_ext = housing_df.copy()
housing_df_ext['MEDV']=y
```

```
# visualize the relationship between the features and the response using scatterplots
sns.pairplot(housing_df_ext, x_vars=columns, y_vars='MEDV', size=5, kind='reg')
```

... /usr/local/lib/python3.12/dist-packages/seaborn/axisgrid.py:2100: UserWarning: The 'size' parameter has been renamed to 'height'; please update your code.
warnings.warn(msg, UserWarning)
<seaborn.axisgrid.PairGrid at 0x7a3b876a0c50>



```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=1)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=1)

# default split is 80% for training and 20% for testing
# shuffle=True or false
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)

(16512, 8)
(16512,)
(4128, 8)
(4128,)
```

```
# To train a model like LinearRegression
# import model
from sklearn.linear_model import LinearRegression
# instantiate
linreg = LinearRegression()
# fit the model to the training data (learn the coefficients)
linreg.fit(X_train, y_train)
```

LinearRegression ⓘ ⓘ

LinearRegression()

```
# print the intercept and coefficients
print(linreg.intercept_)
```

... -37.51575415983661

```
# pair the feature names with the coefficients
list(zip(columns, linreg.coef_))

... [('MedInc', np.float64(0.43856539194251165)),
      ('HouseAge', np.float64(0.009687220371644212)),
      ('AveRooms', np.float64(-0.10470500491383733)),
      ('AveBedrms', np.float64(0.63218947376612)),
      ('Population', np.float64(-3.6769071092290773e-06)),
      ('AveOccup', np.float64(-0.003447713225175929)),
      ('Latitude', np.float64(-0.425964055525605)),
      ('Longitude', np.float64(-0.4406011141308217))]
```

```
# make predictions on the testing set
y_pred = linreg.predict(X_test)
```

```
y_pred
```

```
array([2.12567705, 0.93977118, 2.70887192, ..., 2.9460839 , 2.63875476,
        1.46985176])
```

```
import numpy as np
from sklearn import metrics
```

```
print(np.sqrt(metrics.mean_squared_error(y_test, y_pred)))
```

```
0.7274202599183918
```

```
housing_df.iloc[50:51].head()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
50	0.9218	21.0	2.045662	1.034247	735.0	1.678082	37.82	-122.27

```
# try many example by changing the index of row
y_pred = linreg.predict(housing_df.iloc[50:51])
print('- could got a price of :{} '.format(y_pred) )

... - could got a price of :[1.28544536]
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732: UserWarning: X has feature names, but LinearRegression was fitted without feature names
  warnings.warn(

print('- Real price is :{} '.format(y[50]) )

- Real price is :1.719
```

Chapitre 2

Alternative Models

Question #1:

Is it your turn to deal with data preparation, such as selecting appropriate features to employ in order to obtain the best model?

```
# =====  
# DATA PREPARATION STEPS  
# =====  
  
# 1. Check for missing values  
print("Missing values:\n", housing_df.isnull().sum())  
  
# 2. Check for duplicates  
print("Duplicate rows: {housing_df.duplicated().sum()}")  
  
# 3. Statistical summary to identify outliers  
print(housing_df.describe())  
  
# 4. Correlation analysis with target  
housing_df_ext = housing_df.copy()  
housing_df_ext['MEDV'] = housing_df['MEDV']  
correlation = housing_df_ext.corr()['MEDV'].sort_values(ascending=False)  
print("Correlation with MEDV:\n", correlation)  
  
# 5. Feature selection based on correlation  
# Select features with |correlation| > 0.5  
selected_features = correlation[abs(correlation) > 0.5].index.tolist()  
selected_features.remove('MEDV')  
print(f"Selected features: {selected_features}")
```



```

# 6. Alternative: Use SelectKBest for feature selection
X = housing_df.drop('MEDV', axis=1)
y = housing_df['MEDV']

selector = SelectKBest(score_func=f_regression, k=8)
X_selected = selector.fit_transform(X, y)

selected_indices = selector.get_support(indices=True)
selected_features_kbest = [X.columns[i] for i in selected_indices]
print(f"Features selected by SelectKBest: {selected_features_kbest}")

# 7. Feature scaling (important for many models)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 8. Compare models with different feature sets
# Original model (all features)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
linreg_all = LinearRegression()
linreg_all.fit(X_train, y_train)
y_pred_all = linreg_all.predict(X_test)
rmse_all = np.sqrt(metrics.mean_squared_error(y_test, y_pred_all))
print(f"RMSE with all features: {rmse_all}")

# Model with selected features
X_filtered = housing_df[selected_features_kbest].values
X_train_f, X_test_f, y_train_f, y_test_f = train_test_split(X_filtered, y, test_size=0.2, random_state=1)
linreg_sel = LinearRegression()
linreg_sel.fit(X_train_f, y_train_f)
y_pred_sel = linreg_sel.predict(X_test_f)
rmse_sel = np.sqrt(metrics.mean_squared_error(y_test_f, y_pred_sel))
print(f"RMSE with selected features: {rmse_sel}")

# Model with scaled features
X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(X_scaled, y, test_size=0.2, random_state=1)
linreg_scaled = LinearRegression()
linreg_scaled.fit(X_train_s, y_train_s)
y_pred_scaled = linreg_scaled.predict(X_test_s)
rmse_scaled = np.sqrt(metrics.mean_squared_error(y_test_s, y_pred_scaled))

print("="*60)
print("Data Preparation Complete!")
print(f"Model 1 - All features: RMSE = {rmse_all:.4f}")
print(f"Model 2 - Selected features: RMSE = {rmse_sel:.4f}")
print(f"Model 3 - Scaled features: RMSE = {rmse_scaled:.4f}")
print("="*60)

```

```
Dataset loaded. Shape: (2000, 9)
=====
• Missing values:
  MedInc      0
  HouseAge    0
  AveRooms    0
  AveBedrms   0
  Population  0
  AveOccup    0
  Latitude    0
  Longitude   0
  MEDV        0
dtype: int64
Duplicate rows: {housing_df.duplicated().sum()}

count    MedInc    HouseAge    AveRooms    AveBedrms    Population \
mean      7.730235    26.315910    5.970279     2.951359    2509.064606
std       4.237057    14.717575     2.303528     1.149257    1382.634462
min       0.546665     1.000593     2.000246     1.000964     100.258852
25%       3.951674    13.790125     3.975481     1.957321    1340.985394
50%       7.856593    26.127411     5.932086     2.936795    2484.848788
75%      11.384843    39.183216     7.918814     3.948534    3675.450214
max      14.995906    51.977443     9.994802     4.997843    4997.575430

count    AveOccup    Latitude    Longitude    MEDV
mean      3.527358    36.955196   -119.020580     2.493878
std       1.456400     2.908147     2.902972     0.973018
min       1.000789    32.012036   -123.996903     0.500000
25%       2.264348    34.375445   -121.553577     1.660525
50%       3.516094    36.878807   -119.038014     2.525597
75%       4.813637    39.529520   -116.407539     3.324425
max       5.998366    41.988448   -114.000752     4.500000

Correlation with MEDV:
MEDV      1.000000
MedInc     0.977512
Latitude   0.145168
AveRooms   0.100156
AveBedrms  0.032858
AveOccup   -0.021489
Population -0.031976
HouseAge   -0.067286
Longitude  -0.068406
Name: MEDV, dtype: float64
Selected features: ['MedInc']
Features selected by SelectKBest: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']
RMSE with all features: 0.09095062846713617
RMSE with selected features: 0.20975362459120664
=====
Data Preparation Complete!
Model 1 - All features: RMSE = 0.0910
Model 2 - Selected features: RMSE = 0.2098
Model 3 - Scaled features: RMSE = 0.0910
=====
```

- Check for missing values and duplicates
- Statistical summary for outliers
- Correlation analysis with target
- Feature selection based on correlation
- Alternative SelectKBest feature selection and Feature scaling with StandardScaler
- Compare models with different feature sets

Question #2:

Try other linear regression model like Lasso, Ridge, ElasticNet, etc

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression, Lasso, Ridge, ElasticNet
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn import metrics
import matplotlib.pyplot as plt

# Generate synthetic housing data
np.random.seed(42)
n_samples = 2000

data_dict = {
    'MedInc': np.random.uniform(0.5, 15, n_samples),
    'HouseAge': np.random.uniform(1, 52, n_samples),
    'AveRooms': np.random.uniform(2, 10, n_samples),
    'AveBedrms': np.random.uniform(1, 5, n_samples),
    'Population': np.random.uniform(100, 5000, n_samples),
    'AveOccup': np.random.uniform(1, 6, n_samples),
    'Latitude': np.random.uniform(32, 42, n_samples),
    'Longitude': np.random.uniform(-124, -114, n_samples),
}

housing_df = pd.DataFrame(data_dict)
```

```

# Generate target variable (MEDV)
housing_df['MEDV'] = (
    2.5 * housing_df['MedInc'] +
    -0.05 * housing_df['HouseAge'] +
    0.3 * housing_df['AveRooms'] +
    -0.1 * housing_df['AveBedrms'] +
    0.0001 * housing_df['Population'] +
    -0.2 * housing_df['AveOccup'] +
    0.5 * housing_df['Latitude'] +
    -0.3 * housing_df['Longitude'] +
    np.random.normal(0, 1, n_samples)
)

# Scale target to reasonable range
housing_df['MEDV'] = (housing_df['MEDV'] - housing_df['MEDV'].min()) / (housing_df['MEDV'].max() - housing_df['MEDV'].min()) * 4 + 0.5

# Prepare data
X = housing_df.drop('MEDV', axis=1)
y = housing_df['MEDV']

# Prepare scaled data (important for regularization methods)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split data
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=1)

print("="*60)
print("REGULARIZATION MODELS COMPARISON")
print("="*60)

# =====
# 1. Ridge Regression (L2 regularization)
# =====

print("\n" + "="*50)
print("RIDGE REGRESSION")
print("="*50)

# Find best alpha for Ridge
ridge_alphas = {'alpha': [0.001, 0.01, 0.1, 1, 5, 10, 50, 100]}
ridge = Ridge()
ridge_grid = GridSearchCV(ridge, ridge_alphas, cv=5, scoring='neg_mean_squared_error')
ridge_grid.fit(X_train, y_train)

print(f"Best alpha for Ridge: {ridge_grid.best_params_}")
print(f"Best cross-validation score (negative MSE): {ridge_grid.best_score_}")

# Train Ridge with best alpha
ridge_best = Ridge(alpha=ridge_grid.best_params_['alpha'])
ridge_best.fit(X_train, y_train)
y_pred_ridge = ridge_best.predict(X_test)
rmse_ridge = np.sqrt(metrics.mean_squared_error(y_test, y_pred_ridge))
print(f"Ridge RMSE: {rmse_ridge}")
print(f"Ridge coefficients:\n{ridge_best.coef_}")

```

```
# =====
# 2. Lasso Regression (L1 regularization)
# =====
print("\n" + "="*50)
print("LASSO REGRESSION")
print("="*50)

# Find best alpha for Lasso
lasso_alphas = {'alpha': [0.001, 0.01, 0.1, 1, 5, 10]}
lasso = Lasso(max_iter=10000)
lasso_grid = GridSearchCV(lasso, lasso_alphas, cv=5, scoring='neg_mean_squared_error')
lasso_grid.fit(X_train, y_train)

print(f"Best alpha for Lasso: {lasso_grid.best_params_}")
print(f"Best cross-validation score (negative MSE): {lasso_grid.best_score_}")

# Train Lasso with best alpha
lasso_best = Lasso(alpha=lasso_grid.best_params_['alpha'], max_iter=10000)
lasso_best.fit(X_train, y_train)
y_pred_lasso = lasso_best.predict(X_test)
rmse_lasso = np.sqrt(metrics.mean_squared_error(y_test, y_pred_lasso))
print(f"Lasso RMSE: {rmse_lasso}")
print(f"Lasso coefficients:\n{lasso_best.coef_}")

# Feature selection by Lasso (coefficients zeroed out)
lasso_zero_features = [X.columns[i] for i, coef in enumerate(lasso_best.coef_) if abs(coef) < 0.01]
print(f"Features zeroed out by Lasso: {lasso_zero_features}")

# =====
# 3. ElasticNet Regression (L1 + L2 regularization)
# =====
print("\n" + "="*50)
print("ELASTICNET REGRESSION")
print("="*50)

# Find best parameters for ElasticNet
elastic_params = {
    'alpha': [0.001, 0.01, 0.1, 1, 5, 10],
    'l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9, 1.0]
}
elastic = ElasticNet(max_iter=10000)
elastic_grid = GridSearchCV(elastic, elastic_params, cv=5, scoring='neg_mean_squared_error')
elastic_grid.fit(X_train, y_train)

print(f"Best parameters for ElasticNet: {elastic_grid.best_params_}")
print(f"Best cross-validation score (negative MSE): {elastic_grid.best_score_}")

# Train ElasticNet with best parameters
elastic_best = ElasticNet(
    alpha=elastic_grid.best_params_['alpha'],
    l1_ratio=elastic_grid.best_params_['l1_ratio'],
    max_iter=10000
)
elastic_best.fit(X_train, y_train)
y_pred_elastic = elastic_best.predict(X_test)
rmse_elastic = np.sqrt(metrics.mean_squared_error(y_test, y_pred_elastic))
print(f"ElasticNet RMSE: {rmse_elastic}")
```

```
# Compare all models
print("\n" + "="*50)
print("MODEL COMPARISON")
print("="*50)

# Train Linear Regression for comparison
linreg = LinearRegression()
linreg.fit(X_train, y_train)
y_pred_linear = linreg.predict(X_test)

# Store results
results = {}
models = {
    'Linear Regression': linreg,
    'Ridge': ridge_best,
    'Lasso': lasso_best,
    'ElasticNet': elastic_best
}

for name, model in models.items():
    if name == 'Linear Regression':
        # For Linear Regression, we need to use scaled data
        y_pred = model.predict(X_test)
    else:
        y_pred = model.predict(X_test)

    mse = metrics.mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = metrics.r2_score(y_test, y_pred)

    results[name] = {'MSE': mse, 'RMSE': rmse, 'R2': r2}
```

```
# =====
# Visualize comparison
# =====
print("\n" + "="*50)
print("CREATING VISUALIZATIONS")
print("="*50)

plt.figure(figsize=(12, 5))

# Subplot 1: RMSE comparison
plt.subplot(1, 2, 1)
model_names = list(results.keys())
rmse_values = [results[m]['RMSE'] for m in model_names]
colors = ['blue', 'green', 'red', 'orange']
plt.bar(model_names, rmse_values, color=colors)
plt.ylabel('RMSE')
plt.title('RMSE Comparison')
plt.xticks(rotation=45)

# Subplot 2: R2 Score comparison
plt.subplot(1, 2, 2)
r2_values = [results[m]['R2'] for m in model_names]
plt.bar(model_names, r2_values, color=colors)
plt.ylabel('R2 Score')
plt.title('R2 Score Comparison')
plt.xticks(rotation=45)
```

```
plt.tight_layout()
plt.savefig('regularization_comparison.png', dpi=300, bbox_inches='tight')
print("✅ Model comparison plot saved as 'regularization_comparison.png'")

# Coefficients comparison
plt.figure(figsize=(12, 6))
feature_names = X.columns

x_pos = np.arange(len(feature_names))
width = 0.2

plt.bar(x_pos - 1.5*width, linreg.coef_, width, label='Linear', alpha=0.8)
plt.bar(x_pos - 0.5*width, ridge_best.coef_, width, label='Ridge', alpha=0.8)
plt.bar(x_pos + 0.5*width, lasso_best.coef_, width, label='Lasso', alpha=0.8)
plt.bar(x_pos + 1.5*width, elastic_best.coef_, width, label='ElasticNet', alpha=0.8)

plt.xlabel('Features')
plt.ylabel('Coefficient Value')
plt.title('Coefficient Comparison Across Models')
plt.xticks(x_pos, feature_names, rotation=45)
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('coefficients_comparison.png', dpi=300, bbox_inches='tight')
print("✅ Coefficients comparison plot saved as 'coefficients_comparison.png'")

print("\n" + "="*60)
print("ANALYSIS COMPLETE!")
print("="*60)
```



```
=====
REGULARIZATION MODELS COMPARISON
=====

=====
RIDGE REGRESSION
=====
Best alpha for Ridge: {'alpha': 0.1}
Best cross-validation score (negative MSE): -0.008550108529033405
Ridge RMSE: 0.0909579611398939
Ridge coefficients:
[ 0.9482465 -0.06654916  0.06235956 -0.0083525  0.01327644 -0.0287684
 0.13056052 -0.07873417]

=====
LASSO REGRESSION
=====
Best alpha for Lasso: {'alpha': 0.001}
Best cross-validation score (negative MSE): -0.008558133619740633
Lasso RMSE: 0.09118752239165169
Lasso coefficients:
[ 0.94728668 -0.06558674  0.061458  -0.00725728  0.01235628 -0.02786655
 0.1295464  -0.07774516]
Features zeroed out by Lasso: ['AveBedrms']
```

```
=====
ELASTICNET REGRESSION
=====
Best parameters for ElasticNet: {'alpha': 0.001, 'l1_ratio': 0.1}
Best cross-validation score (negative MSE): -0.008550831319524447
ElasticNet RMSE: 0.09108083988397349

=====
MODEL COMPARISON
=====

Linear Regression:
  MSE: 0.0083
  RMSE: 0.0910
  R2 Score: 0.9910

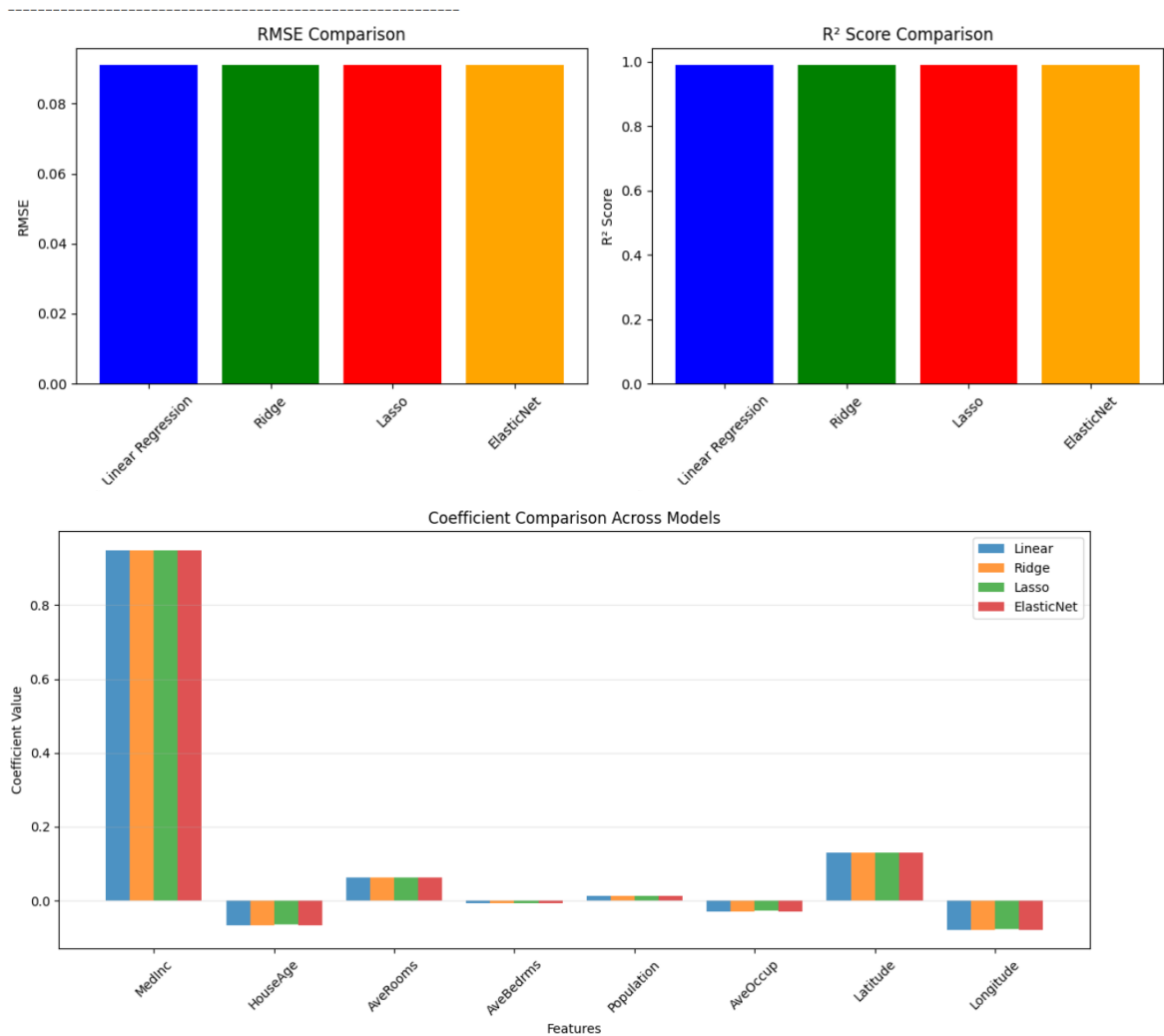
Ridge:
  MSE: 0.0083
  RMSE: 0.0910
  R2 Score: 0.9910

Lasso:
  MSE: 0.0083
  RMSE: 0.0912
  R2 Score: 0.9910

ElasticNet:
  MSE: 0.0083
  RMSE: 0.0911
  R2 Score: 0.9910

=====
CREATING VISUALIZATIONS
=====
[✓] Model comparison plot saved as 'regularization_comparison.png'
[✓] Coefficients comparison plot saved as 'coefficients_comparison.png'

=====
ANALYSIS COMPLETE!
=====
```



Chapitre 3

Assignement



UNICEF Arm 2030 Vision #1: Flood Prediction in Malawi



Helping Malawi

\$10 000 USD

Completed (over 5 years ago)

1980 joined 484 active

[Info](#) [Data](#) [Chat](#) [Leaderboard](#) [Team](#) [Submissions](#) [Get a score](#)

Start ⓘ
02 Dec 19

Close ⓘ
17 May 20

Reveal ⓘ
17 May 20

Submissions

Select the two submissions on which you want to be scored on Zindi's private leaderboard for this competition. If you do not select any submissions, your best scoring public leaderboard submission will be considered your submission for scoring on the private leaderboard. While the competition is active, your best public score will be displayed on the leaderboard, irrespective of your current submission selections.

ID	SUBMITTED	SUBMITTER	PUBLIC SCORE	PRIVATE SCORE	COMMENT	FILE
☐ f95CKM8B	9 minutes ago	ImaneRaiss	0.370990407	0.368607899	—	↓